HouseLens model training and testing

This notebook follows the sample submission: inspect the data, prepare inputs, split into training and test groups, compare models, and review results. The data is historical house sales in Ames, Iowa, USA. Prices are in US dollars.

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from train_model import build_preprocessor, FEATURES

pd.set_option('display.max_columns', 20)
```

1. Load and inspect the training data

The target column is `SalePrice`. The Kaggle competition test file has no target values, so it cannot be used to calculate model accuracy.

In [2]:
```
data = pd.read_csv('data/train.csv')
print('Rows and columns:', data.shape)
print('Sale price range:', data['SalePrice'].min(), 'to', data['SalePrice'].max(),
display(data[['OverallQual', 'GrLivArea', 'Neighborhood', 'SalePrice']].head())
print('Missing values in selected features:')
display(data[FEATURES].isnull().sum().to_frame('Missing values'))
```

```
Rows and columns: (1460, 81)
Sale price range: 34900 to 755000 USD
```

	OverallQual	GrLivArea	Neighborhood	SalePrice
0	7	1710	CollgCr	208500
1	6	1262	Veenker	181500
2	7	1786	CollgCr	223500
3	7	1717	Crawfor	140000
4	8	2198	NoRidge	250000

```
Missing values in selected features:
```

	Missing values
OverallQual	0
GrLivArea	0
GarageCars	0
TotalBsmtSF	0
FullBath	0
YearBuilt	0
LotArea	0
Neighborhood	0

2. Prepare features and make a holdout split

Eighty percent is used to fit the comparison models. The remaining 20% is kept aside to check predictions on rows those models did not train on. A fixed random seed makes the split repeatable.

In [3]:
```
X = data[FEATURES]
y = data['SalePrice']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
print('Training rows:', len(X_train))
print('Holdout rows:', len(X_test))
```

```
Training rows: 1168
Holdout rows: 292
```

3. Train and compare three models

The median baseline gives context: a machine-learning model should do better than always predicting a typical price. The preprocessing fills missing values and converts neighborhood names into columns.

In [4]:
```
models = {
    'Median price baseline': Pipeline([
        ('prepare', build_preprocessor()),
        ('model', DummyRegressor(strategy='median'))
    ]),
    'Linear regression': Pipeline([
        ('prepare', build_preprocessor(scale=True)),
        ('model', LinearRegression())
    ]),
    'Random forest': Pipeline([
        ('prepare', build_preprocessor()),
```

```
        ('model', RandomForestRegressor(
            n_estimators=150, max_depth=12, min_samples_leaf=2,
            random_state=42, n_jobs=-1
        ))
    ])
}

results = []
for name, model in models.items():
    model.fit(X_train, y_train)
    predicted = model.predict(X_test)
    results.append({
        'Model': name,
        'MAE (USD)': mean_absolute_error(y_test, predicted),
        'RMSE (USD)': np.sqrt(mean_squared_error(y_test, predicted)),
        'R2': r2_score(y_test, predicted)
    })

results = pd.DataFrame(results).sort_values('MAE (USD)')
display(results.round(2))
best_name = results.iloc[0]['Model']
print('Best by holdout MAE:', best_name)
```

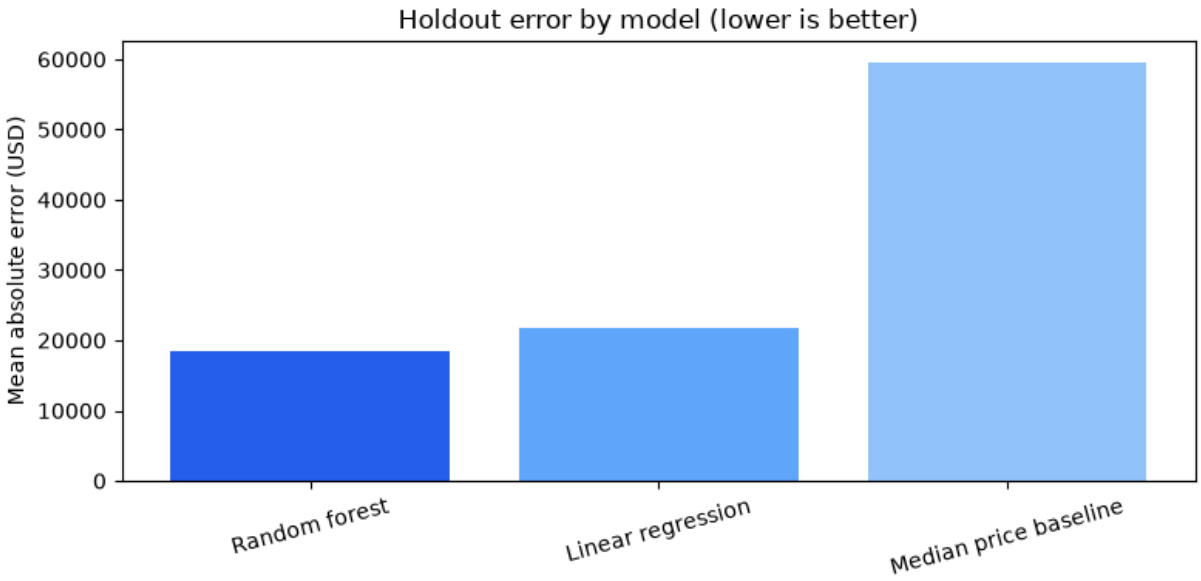
	Model	MAE (USD)	RMSE (USD)	R2
2	Random forest	18529.55	29695.05	0.89
1	Linear regression	21842.99	36228.29	0.83
0	Median price baseline	59568.25	88667.17	-0.02

```
Best by holdout MAE: Random forest
```

4. Compare errors visually

MAE is the average absolute difference between a predicted and actual price. A lower value is better.

In [5]:
```
plt.figure(figsize=(8, 4))
plt.bar(
    results['Model'], results['MAE (USD)'],
    color=['#2563eb', '#60a5fa', '#93c5fd']
)
plt.ylabel('Mean absolute error (USD)')
plt.title('Holdout error by model (lower is better)')
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```



5. Review some random forest predictions

The error for one house can be much higher or lower than the overall MAE. The MAE is only an average for this holdout sample.

In [6]:
```
forest = models['Random forest']
example_predictions = pd.DataFrame({
    'Actual sale price': y_test.iloc[:10].values,
    'Predicted price': forest.predict(X_test.iloc[:10])
})
example_predictions['Absolute error'] = (
    example_predictions['Actual sale price'] -
    example_predictions['Predicted price']
).abs()
display(example_predictions.round(0))
```

	Actual sale price	Predicted price	Absolute error
0	154500	137674.0	16826.0
1	325000	331985.0	6985.0
2	115000	116648.0	1648.0
3	159000	154185.0	4815.0
4	315500	311278.0	4222.0
5	75500	79282.0	3782.0
6	311500	207733.0	103767.0
7	146000	152431.0	6431.0
8	84500	78434.0	6066.0
9	135500	133113.0	2387.0

6. Use the separate Kaggle test file

The labeled training set is used for the holdout evaluation above. Kaggle's separate test file has no actual `SalePrice`, so its predictions are saved for later use but cannot be scored here. The HouseLens website model is refit on all labeled rows by `train_model.py`.

Conclusion

On this fixed holdout split, the selected model can estimate Ames sale prices more closely on average than the median baseline. The data is historical and from one US city. This project does not establish performance in the Netherlands, today's market, or for unusual homes. The estimate is decision support, not financial advice.

In [7]:
```
competition_test = pd.read_csv('data/test.csv')
print('Unlabeled Kaggle test rows:', len(competition_test))
test_predictions = pd.read_csv('reports/test_predictions.csv')
display(test_predictions.head())
```

```
Unlabeled Kaggle test rows: 1459
```

	Id	SalePrice
0	1461	127049.435198
1	1462	155121.987136
2	1463	173110.022887
3	1464	180406.545595
4	1465	202440.764648

HouseLens project supplement

A beginner machine-learning project for house price estimation

Project purpose

HouseLens explores whether a small machine-learning model can estimate a historical house sale price from a few property details. A simple website lets a person change the details, see an estimate, compare past prices in an Ames neighborhood, and review how the model performed. It supports understanding and comparison; it does not decide whether a house is a good purchase.

Research question

How accurately can beginner-friendly regression models estimate historical Ames house sale prices from selected property characteristics, and how should the website explain uncertainty and limitations?

Supporting questions

- Which selected property details have the strongest relationship with the estimate?
- Does machine learning improve on a simple median-price guess for unseen rows?
- What limits should be visible to a website user?

Dataset and scope

The supplied files come from Kaggle House Prices: Advanced Regression Techniques. The labeled training file contains 1,460 Ames, Iowa, sales with SalePrice as the target. The separate test file has 1,459 rows without sale prices. Sales cover 2006 to 2010, and app prices are historical US dollars. This is not current Dutch market data.

The website model uses eight inputs: overall quality, above-ground living area, garage capacity, basement area, full bathrooms, year built, lot size, and neighborhood. This smaller set keeps the first version understandable, but leaves out many details that affect real prices.

Data source: Kaggle House Prices: Advanced Regression Techniques.

Project workflow

Step	What happens
1. Inspect	Load the supplied CSV files and check the target and selected inputs.
2. Prepare	Fill missing numeric values with the median, fill missing neighborhoods with the most common category, and one-hot encode neighborhood names.
3. Split	Set aside 20% of labeled rows for evaluation. Use a fixed random seed so the split can be repeated.
4. Compare	Train a median baseline, linear regression, and random forest. Compare errors on the same held-out rows.
5. Use the model	Refit the selected model on all labeled rows and load it into the Streamlit website.

Training and evaluation results

The labeled data was divided into 1,168 training rows and 292 holdout rows. The holdout rows were kept out of model fitting so they could provide a fairer estimate of performance on houses the model had not seen during training. Each comparison model used the same split.

Model	MAE (USD)	RMSE (USD)	R2
Random forest	$18,529.55	$29,695.05	0.8850
Linear regression	$21,842.99	$36,228.29	0.8289
Median-price baseline	$59,568.25	$88,667.17	-0.0250

How to read the scores

MAE is the average absolute difference between predicted and actual prices. It is easiest to read in dollars, and lower is better. The random forest has a holdout MAE of about $18,530, which means its predictions differed from the actual prices by that amount on average for this one holdout sample. It is not a promise that any individual estimate will be within $18,530.

RMSE also measures prediction error, but gives more weight to large misses. **R2** describes how much variation the model captures compared with a simple average-based reference; higher is generally better. The median baseline performs worse than the other two models on this split, which gives evidence that the selected property details contain useful price information.

The random forest was selected because it had the lowest holdout MAE. The training script used 150 trees, a maximum tree depth of 12, a minimum of two rows per leaf, and a fixed random seed. After comparison, it was refit on all 1,460 labeled rows for the website. The Kaggle test.csv file has no SalePrice labels, so its 1,459 predictions are saved separately and are not included in the evaluation scores.

The model comparison is based on one random split. Repeating the evaluation with cross-validation or testing against newer data would give stronger evidence. The comparison does not show that the model will work for another city or year.

Responsible design and project reflection

The Potential Impact Assessment describes HouseLens as decision support. The website applies that principle in a few practical ways:

User control. A person chooses the property details and neighborhood. HouseLens displays an estimate and does not label a house as a good or bad purchase.

Transparency. The app names the Ames data and US dollar currency, shows the holdout error and model comparison, and separates a model estimate from the historical price chart.

Privacy. This prototype has no accounts and does not store names, budgets, addresses, or user preferences. Its input values are used to make a prediction in the running app.

Fairness and coverage. The data comes from one city, and neighborhoods can have very different numbers of examples. The model can therefore be less reliable in areas or for homes that are poorly represented. Historical sales also reflect past market patterns, including any inequities in those patterns.

Uncertainty. The website shows the holdout MAE as a rough reference around the estimate. It clearly says this is not a formal prediction interval or a guaranteed error range. One sample prediction in the notebook differs from its actual sale price by over $100,000, showing why the average should not be treated as a certainty.

Efficient computation. The project uses a small tabular dataset and a conventional random forest instead of a large generative model. This is enough to test the prediction question without unnecessary computation.

Limitations

The data is historical and limited to Ames, Iowa. It covers sales from 2006 to 2010 and cannot support a claim about today's market or the Netherlands. The model uses only eight features and leaves out condition, renovation quality, changing interest rates, current supply and demand, and other relevant information. One holdout split may produce different scores from another split. The model can also make large errors for unusual properties.

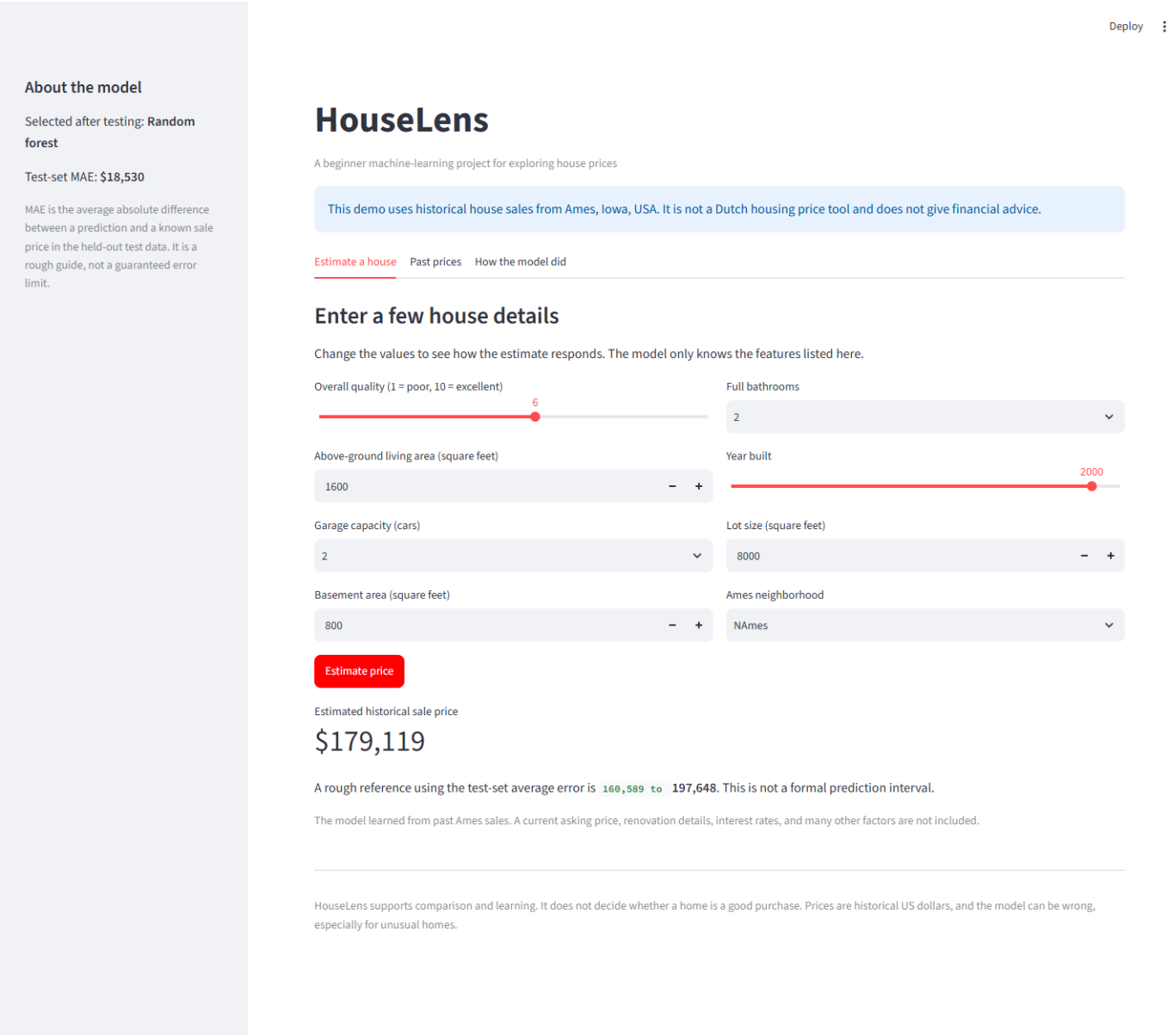
What I would improve next

I would compare the models over repeated splits or cross-validation, inspect more of the largest prediction errors, and ask potential users whether the estimate and uncertainty explanation are understandable. Before making a Dutch housing version, I would need current and appropriately licensed Dutch data and a new evaluation. A production service would also need a fuller review of data rights, privacy, fairness, and legal responsibilities.

Conclusion

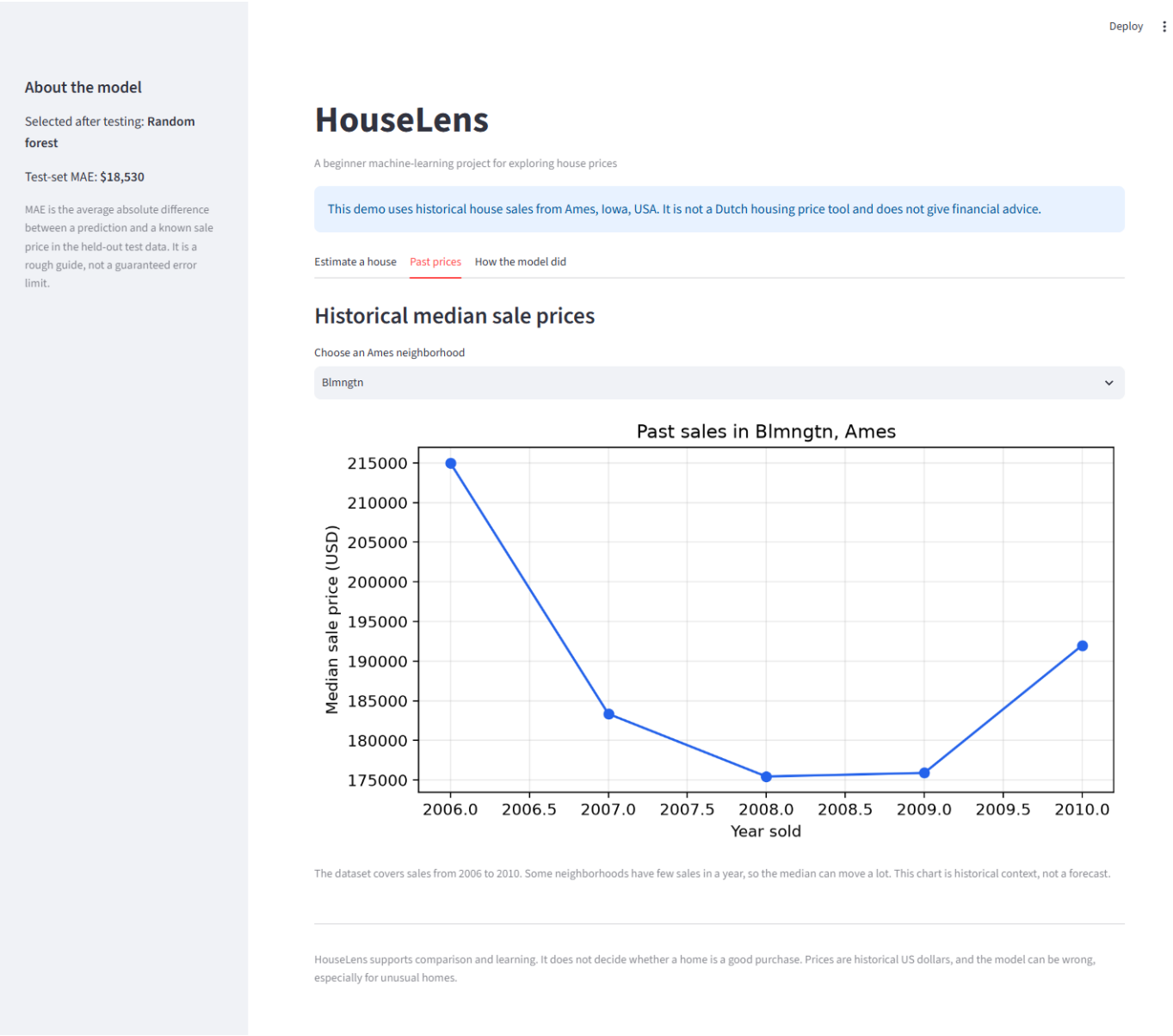
In this experiment, the random forest estimated held-out Ames sale prices more closely on average than linear regression or the median baseline. HouseLens makes that result explorable through a small website and explains where it can fail. The evidence supports the project as a learning and comparison tool; it is not sufficient for a real house-buying decision.

Website view: house estimate



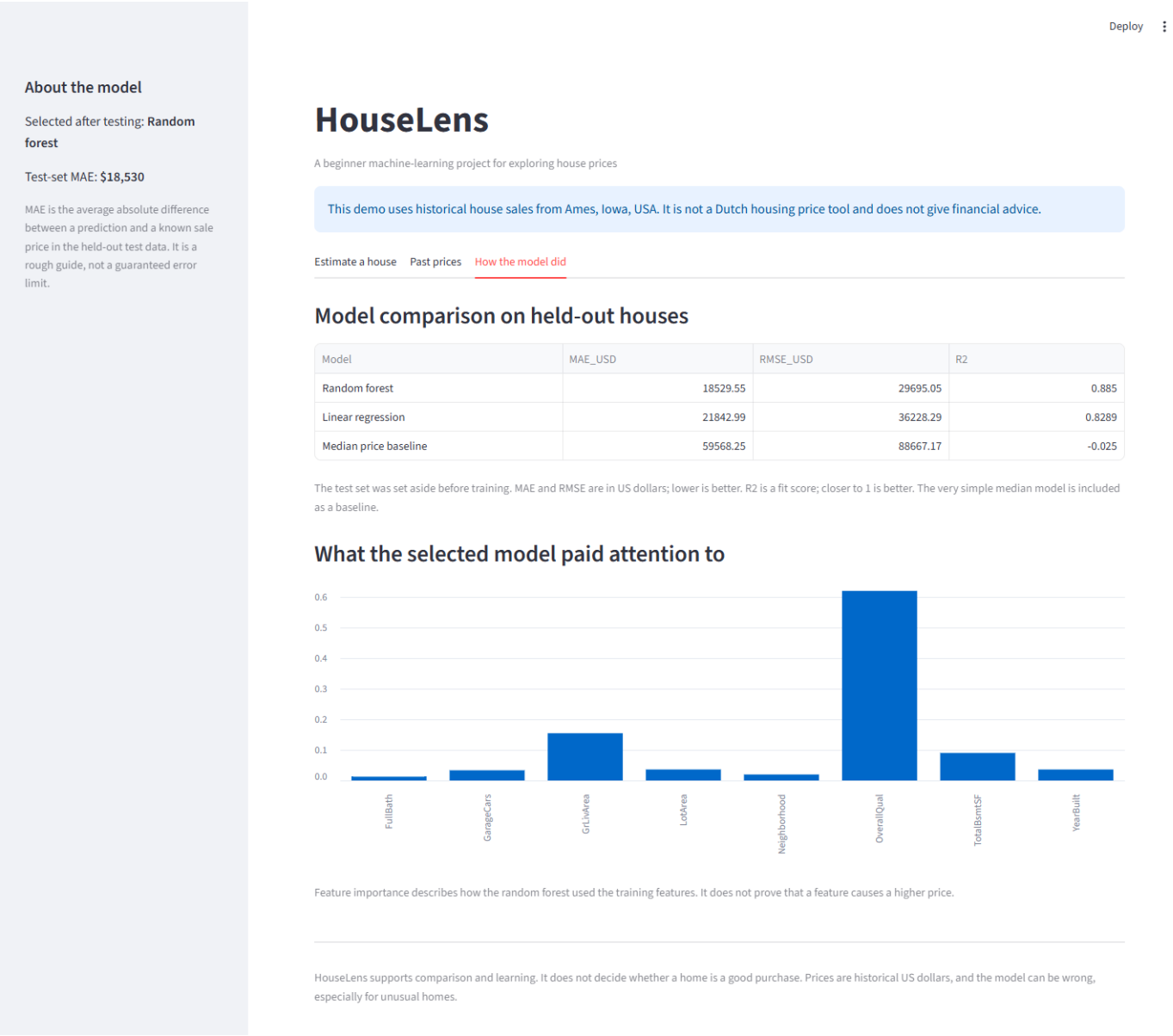
The form asks for eight property details and shows a sample estimate. In this captured example, the model returns $179,119. The displayed range uses average holdout MAE as a rough reference, not as a formal prediction interval.

Website view: past prices



This chart shows historical median prices for the selected Ames neighborhood from 2006 to 2010. The interface explains that sparse sales can make yearly medians move and that the chart is context, not a forecast.

Website view: model results



The comparison page reports the held-out MAE, RMSE, and R2 scores. Its feature-importance chart describes how the random forest used the inputs; it does not prove cause and effect.